

Jenkins

Jenkins es un servidor de automatización de código abierto escrito en Java que sirve para implementar flujos de integración continua y entrega continua (CI/CD) en el desarrollo de software.

¿Qué es Jenkins?

Es una herramienta multiplataforma y gratuita que actúa como el motor central para automatizar las tareas repetitivas del ciclo de vida de un programa. Permite conectar repositorios de código con herramientas de prueba y despliegue a través de una interfaz web o mediante código (pipelines).

¿Para qué sirve?

Sirve para automatizar todo el proceso que va desde que un desarrollador escribe código nuevo hasta que la aplicación queda lista o instalada en producción.

Sus funciones principales son:

- **Detectar cambios:** Revisa de forma automática los repositorios de código (como GitHub o GitLab) cuando subes una modificación.
- **Compilar:** Traduce o empaqueta el código fuente para generar los archivos ejecutables o builds.
- **Probar:** Ejecuta pruebas unitarias y de integración para comprobar que no hay errores ni fallos.
- **Desplegar:** Envía la aplicación ya probada a servidores de prueba, ensayo o producción.
- **Notificar:** Alerta de inmediato al equipo de desarrollo si una compilación o prueba falla.
- **Extender funciones:** Cuenta con cientos de plugins para integrarse con Docker, Maven, Slack y otras tecnologías.

Proceso detallado de un flujo en Jenkins

Jenkins automatiza el ciclo de vida del software mediante un flujo secuencial llamado pipeline que compila, prueba y despliega código de forma repetible.

1. **Detección de cambios (Trigger):** El proceso inicia cuando un desarrollador sube un cambio (commit) a un repositorio de código fuente como Git o GitHub. Jenkins detecta este evento mediante un gancho (webhook) o revisiones periódicas.
2. **Descarga del código (Checkout):** Jenkins se conecta al repositorio y clona o descarga la versión más reciente del código fuente para iniciar la tarea.
3. **Construcción (Build):** Se compila el código y se preparan los binarios o dependencias utilizando herramientas integradas como Maven, Gradle o npm.
4. **Pruebas (Test):** Se ejecutan pruebas automatizadas (unitarias o de integración) con frameworks como [JUnit](#) para validar que el nuevo código no rompa la aplicación.
5. **Generación de artefactos:** Si las pruebas son exitosas, se empaqueta el resultado (por ejemplo, creando una imagen de Docker) listo para su distribución.
6. **Despliegue (Deploy):** El sistema transfiere e instala la aplicación en un entorno de pruebas (staging) o producción de forma automática o controlada.

7. **Notificación y registros (Logs):** Jenkins registra métricas de tiempo, fases y resultados. Si el proceso falla, avisa de inmediato al equipo por correo o canales como Slack.

Jenkins Pipeline

Un pipeline declarativo básico en Jenkins se compone de:

- **Un agente**
- **Etapas (stages)**
- **Pasos (steps)**

que ejecutan tareas secuenciales.

Se puede consultar la documentación oficial en los [Ejemplos de Pipeline de Jenkins](#) para ver casos más avanzados.

Ejemplo de Código (Jenkinsfile)

Este script muestra una estructura típica con tres fases: compilar, probar y desplegar:

[groovy](#).

```
pipeline {
    agent any

    stages {
        stage('Compilar') {
            steps {
                echo 'Iniciando la compilación...'
                sh 'echo "Compilando el código fuente"'
            }
        }

        stage('Pruebas') {
            steps {
                echo 'Ejecutando pruebas unitarias...'
                sh 'echo "Pruebas exitosas"'
            }
        }

        stage('Despliegue') {
            steps {
                echo 'Desplegando la aplicación...'
                sh 'echo "Despliegue completado con éxito"'
            }
        }
    }
}
```

Pasos para crear un Pipeline en Jenkins

- Ir a la interfaz principal de Jenkins y haz clic en Nueva Tarea (New Item).

- Escribir el nombre del proyecto y seleccionar la opción Pipeline. Hacer clic en OK.
- Desplazarse a la sección inferior llamada Pipeline.
- Pegar el código de ejemplo anterior en el cuadro de texto Script.
- Hacer clic en Guardar (Save) y luego en Construir ahora (Build Now) para ver el flujo de ejecución.

[Un caso de uso del Pipeline de Jenkins](#)

Un caso real

Un caso real de uso de Jenkins es la automatización del proceso de compilación, prueba y despliegue (CI/CD) para una aplicación web basada en Node.js alojada en un repositorio de GitHub.

El Problema

Los desarrolladores subían cambios al repositorio de Git constantemente. Hacer el proceso de pruebas, empaquetado y actualización del servidor de producción de forma manual consumía mucho tiempo y causaba errores humanos frecuentes.

La Solución con Jenkins

Se configuró una tubería automatizada o pipeline mediante un archivo llamado Jenkinsfile dentro del mismo proyecto. El flujo real opera de la siguiente manera:

1. **Detección de cambios (Trigger):** Un desarrollador hace un git push a la rama principal (main) en el repositorio.
2. **Clonación (Sources):** Jenkins detecta el cambio automáticamente, se conecta al repositorio y descarga la última versión del código fuente.
3. **Construcción y Dependencias (Build):** El servidor ejecuta comandos para preparar el entorno, como la instalación de dependencias mediante npm install.
4. **Pruebas (Test):** Se ejecutan pruebas automatizadas para verificar que el nuevo código no rompa las funciones existentes. Si las pruebas fallan, el proceso se detiene y Jenkins avisa al equipo por correo o chat.
5. **Despliegue (Deploy):** Si todo es correcto, se conecta por SSH al servidor remoto, sincroniza los archivos nuevos y reinicia el servicio con un comando como npm start para que los cambios estén disponibles para los usuarios en línea.